Two formalizations of Scott's approximation layer, compared

Ours — `ScottDomains/`, formalizing C. A. Gunter and D. S. Scott, *Semantic Domains* (HTCS Vol. B, 1990).

Theirs — `scott1972/`, Lars Warren Ericson, *A Lean 4 Formalization of Scott's Continuous Lattices (1972)*, arXiv **2606.30782**, formalizing D. S. Scott, *Continuous Lattices* (1972). Cloned at `../../scott1972`; the paper is at `../papers/Ericson 2026 … arXiv-2606.30782.pdf`.

Both are `sorry`-free and both check on `[propext, Classical.choice, Quot.sound]`. Verified here rather than read off the abstracts: `scripts/scott1972-verify.sh` (936 jobs, exit 0, zero `sorry` in the sources) and `scripts/scott1972-axioms.sh` on Theorem 4.4's two halves.

1. They are not the same subject

The two papers are eighteen years apart and sit on opposite sides of one distinction.

#		Scott 1972 (theirs)	Gunter & Scott 1990 (ours)
1	Carrier	**complete lattice**	**cpo** — directed sups, ⊥, no top, no meets
2	Approximation	**continuous**: `y = ⊔{x ∣ x ≪ y}`	**algebraic**: `y = ⊔{k ∈ K(D) ∣ k ⊑ y}`, `k` compact
3	Entry point	**topological** — injective T_0 spaces, Scott-open sets	**order-theoretic** — directed sets and least upper bounds
4	`D ≅ [D → D]` via	the **inverse limit** `D∞`	Theorem 21 + Lemma 23, **no inverse limit at all**
5	Countability	not required	**required** — `Domain` carries `countable_compacts`

Row 2 is a containment, not a disagreement: every algebraic lattice is a continuous lattice, and `x ≪ x` is exactly compactness. Row 4 is the sharpest divergence — `PaperInventory.md` records that §7 of the 1990 paper builds no inverse limit, and an early draft of that row listing `D∞` as an outstanding definition was struck for exactly that reason. **So this artifact formalizes the construction our paper deliberately declines to use.**

2. ≪ is defined differently in the two developments

Same symbol, same notation ≪, different definitions and different settings.

Theirs (`Scott1972/ContinuousLattice/WayBelow.lean:72`), topological, on a `CompleteLattice`:

```
def WayBelow (x y : D) : Prop :=
  ∃ U : Set D, ScottOpen U ∧ y ∈ U ∧ U ⊆ Set.Ici x
```

`y` lies in the interior of the principal up-set of `x` for the Scott topology, where `ScottOpen U` is `IsUpperSet U` plus inaccessibility by directed suprema.

Ours (`ScottDomains/WayBelow.lean:51`), order-theoretic, on a `Preorder`:

```
def WayBelow [Preorder α] (x y : α) : Prop :=
  ∀ (s : Set α) (u : α), s.Nonempty → DirectedOn (· ≤ ·) s → IsLUB s u → y ≤ u →
    ∃ z ∈ s, x ≤ z
```

Three consequences worth stating.

1. **Ours needs no suprema.** It is stated at `[Preorder α]` and quantifies over `IsLUB` rather than applying `sSup`, so it is available before any completeness is assumed. Theirs needs `CompleteLattice` for `sSup` and for the Scott topology to exist.
2. **Ours makes compactness definitional.** `x ≪ x ↔ IsCompactElement x` holds by `Iff.rfl`. Theirs has no compact elements at all — continuity, not algebraicity, is the 1972 notion.
3. **The two agree where both apply** (a complete lattice), by the standard equivalence between the topological and order-theoretic readings, but **neither development proves that** — and neither could import the other, see §5.

3. Size

#	Layer	Theirs	Ours
1	way-below / approximation	19 thms, 236 lines	7 thms, 117 lines
2	the approximation class	`IsContinuousLattice`, in the same file	`IsAlgebraic`, `Domain`, `BoundedComplete` — 13 thms, 227 lines
3	function spaces	**110 thms, 1633 lines**	`ScottHom+StepFunction+FunctionSpaceDomain+CompactFunction+FunctionSpa` — 41 thms, 887 lines
4	inverse limits / `D∞`	47 + 33 thms, 1241 lines	**none**
5	whole development	**273 theorems**	1298 theorems

Row 3 is the striking one: their function-space layer is nearly twice ours in theorems and lines, for a smaller stated result. The reason is row 3 of §1 — working topologically means every continuity argument passes through Scott-open sets and the `⇑a` basis, where ours works directly with directed sets and step functions.

Row 5 is not a quality comparison. Theirs formalizes 43 numbered results of one paper; ours formalizes 24 of 29 numbered results plus 91 prose claims of a different, longer paper, and carries powerdomains, bifinite domains and §7's universal domains that Scott 1972 does not contain.

4. What each has that the other does not

Theirs, absent from ours:

- **Injective T_0 spaces** (`Injective.lean`) and the Sierpiński space as `Prop` — Scott's topological characterization. We have no topology anywhere.
- **The Scott topology as a topology** — `ScottOpen`, closed under $\cap$ and $\bigcup_0$. Ours never constructs it; `ExistingTheories.lean` only `#check`s Mathlib's `Topology.IsScott` without using it.
- **Inverse limits and `D∞`** — `InverseLimits.lean`, `FunctionSpaceTower.lean`, and the capstone `D∞ ≅ [D∞ → D∞]`.
- **The Milner correction** (`MilnerCorrection.lean`) — the March 1972 erratum without which Propositions 2.9, 2.10 and 3.3 are wrong as printed. A formalization catching a printed error is a pattern this project knows well: ours records the same for Lemma 9's items 3 and 5, Theorem 16's second conjunct, and three defects in §7.4's printed pre-ordering.

Ours, absent from theirs:

- **Compact elements and algebraicity** — `IsCompactElement`, `compacts`, `compactsBelow`, and the countability condition that makes a `Domain`.
- **Powerdomains** — Hoare, Smyth and Plotkin, with Theorem 12's initiality.
- **Bifinite domains**, the Plotkin order, and finitary projections.
- **§7's universal domains** — `Dyadic.U`, `Colimit.V`, representability and p-representability.
- **Bounded completeness as a separate predicate**, which is what lets §6 drop it deliberately.

5. Why nothing can be shared mechanically

The artifact pins `leanprover/lean4:v4.30.0`; this project pins **v4.32.2**. They therefore carry two independent Mathlib checkouts — about 7 GB each, unable to share oleans — and **no import can cross between them**. Any transfer is a deliberate port of a definition or proof, read and rewritten, not an `import`.

Two further obstacles to reuse, both of which also apply to our own submission plans:

1. **Naming.** Both developments name after the source paper's numbering — theirs `Theorem212`, ours `thm11`, `lem23`, `prop15`. Mathlib names describe the statement, so both would need the same rename before upstreaming.
2. **Carrier mismatch.** Their results are stated for `CLat`, a bundled complete lattice. Ours are stated for an unbundled `[CompletePartialOrder α]` with `[IsAlgebraic α]`/`[Domain α]` as separate classes. Porting a theorem means re-deriving it at the weaker carrier, not translating it.

6. What this is useful for

1. **A second opinion on $\ll$.** Two independent Lean definitions of the same relation, from two papers, is the strongest available check that ours says what it should. Proving them equivalent on a complete lattice would be a real result and is

not in either development.
2. **The road not taken.** If the `D∞` construction is ever wanted here — it is not needed for Gunter & Scott, which is the point — this is a worked, kernel-checked route.
3. **A calibration for the audit.** r0038 measured our speculative-API rate at ≈3.5%. Running the same measurement over 273 theorems formalizing one paper would say whether that figure is a property of this project or of the genre.

7. Reproducing the measurements here

```
scripts/scott1972-verify.sh all     # cache fetch + lake build Scott1972
scripts/scott1972-axioms.sh         # #print axioms on Theorem 4.4's two halves
scripts/lean-decls.py --per-file <files>
```

All figures in this document come from those, run on 2026-08-08 against artifact `HEAD` and this project at commit `a8474d3`.